

- They provide seamless integration between isolated services.
- The microservices design can be easily integrated with cloud environments on cloud native containers or over cloud based virtual machines.
- Microservices can be built using open standards such as REST, JSON, OAuth and others. They can be easily integrated with any web or mobile client.
- Microservices can be leveraged to create an extensible solution by quickly onboarding newer modules.
- They help to build large and complex systems with high fault tolerance.
- This provides the flexibility to change the tools and technologies without impacting the business functionality.
- The architecture provides on-demand scalability and high availability.
- They help to adopt DevOps principles for continuous integration and continuous delivery with faster release time.
- A microservices layer can be built on top of traditional SOA-based web services in legacy platforms to implement a multi-speed IT model.
- Microservices can be managed centrally with very little effort.

Microservices reference architecture

The key layers and components of a typical microservice architecture are depicted in Figure 1.

A user experience layer should support many channels and a variety of devices. To support diversification, the user interface layer is usually built using JavaScript frameworks such as ReactJS or Angular that are responsive and lightweight. Single page applications are widely used for developing IT service frontends. For example, virtual workspaces, self-service requests, desktop provisioning, monitoring views, capacity reports, billing views, and trigger specific admin tasks can all be implemented

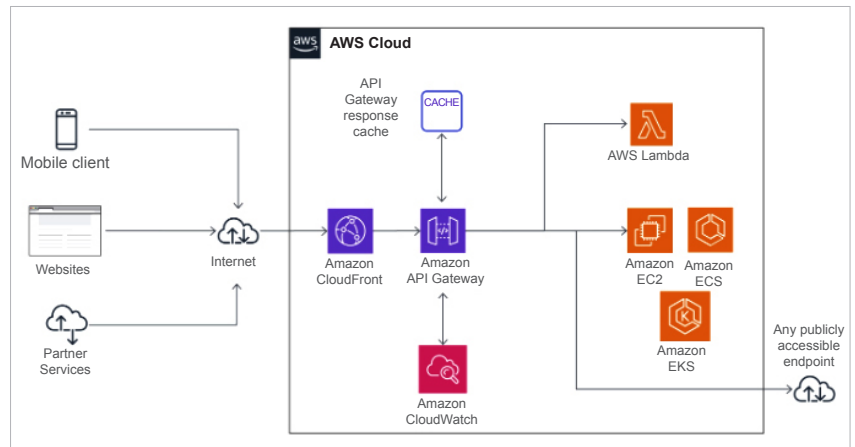


Figure 2: Cloud native microservices illustration on AWS

as single page applications. Backend services serving them are completely isolated and can span multiple platforms. We can integrate backend services with REST invocations using JSON payload structures. Business services can natively support APIs or we can develop a microservices wrapper for legacy applications. Lightweight token-based security can be used for authentication and authorisation.

API Gateway is the proxy for client apps to access microservices. Any transformations such as payload or protocol take place at this layer. Service discovery acts as directory service. Load balancer is responsible for routing. Circuit breaker handles service failures and fall backs.

Support layer provides DevOps and other functions. All these layers and modules can be implemented on cloud or on-premises infrastructure. A container ecosystem is widely used for microservices deployments.

Cloud native microservices

Public clouds provide the necessary tools, services and components to build the microservices. Figure 2 shows an AWS cloud native microservices setup. API Gateway handles API calls and interacts with other components. Requests from mobile devices, websites, or other channels are routed to the closest CloudFront point of presence (PoP) to minimise latency and provide optimum user experience.

Open source tools

There are many open source tools available that can be leveraged in microservices development and deployment. Open source API gateways such as Apigee, Apiman, API Umbrella, 3scale, Gravitee, and Kong are useful. Many open source container platforms are available such as Kubernetes, OpenShift, and Nomad. For CI/CD, we can make use of Jenkins, GitLab, Buildbot, etc. For monitoring, open source tools such as Grafana, Graphite, Nagios and OpenNMS can be leveraged. **END** 🐧

By: Narasimha Sekhar Kakaraparthi

The author has more than 27 years of experience in software product development. He is currently working as principal architect for virtual desktops and end user services at Wipro Technologies. He is a senior member of Wipro's eminent architects group called distinguished member of technical staff (DMTS).

Disclaimer: The views expressed in this article are that of the author and Wipro does not subscribe to their substance, veracity or truthfulness.